

KIE2010 - MICROPROCESSOR AND MICROCONTROLLER, ASSIGNMENT 1

AYDAN KHAIRIN BIN KHAIRUL ZAMAN, 23002155/2

April 2026

Contents

1	BCD to Binary Conversion	3
1.1	Description	3
1.2	Explanation	3
1.3	Execution Result	6
2	ASCII to BCD Conversion	7
2.1	Description	7
2.2	Explanation	7
2.3	Execution Result	10
3	Packed BCD to ASCII Conversion	11
3.1	Description	11
3.2	Explanation	11
3.3	Execution Result	14
4	Binary to Packed BCD Conversion	15
4.1	Description	15
4.2	Explanation	15
4.3	Execution Result	18
	References	19

1 BCD to Binary Conversion

1.1 Description

This ARM assembly program converts a 2-digit binary coded decimal (BCD) number into its equivalent 8-bit binary value using the formula: $\text{Binary} = (\text{tens} \times 10) + \text{ones}$

1.2 Explanation

```
30
31  main
32      push {lr}
33      ldr r1, =0x20000000 ;tens
34      ldr r2, =0x20000010 ;ones
35      ldr r3, =0x20000020 ;data
36      ldr r4, =test_data
37
38  loop_tests
39      ldrb r5, [r4], #1
40      ldrb r6, [r4], #1
41
42      strb r5, [r1]
43      strb r6, [r2]
44
45      bl test_case
46      add r1, r1, #0x01
47      add r2, r2, #0x01
48      strb r7, [r3], #0x01
49      sub r0, r0, #1
50      cmp r0, #0
51      bne loop_tests
52
53      pop {lr}
54      bx lr
55
```

Figure 1.1: Main loop to iterate through different test cases and calls logic function

Below is the memory layout which store that will store the output for visualisation and debugging based on the code from lines 32 to 35 from Figure 1.1

- Tens digits stored at R1 at location 0x20000000
- Ones digits stored at R2 at location 0x20000010

- Binary output value stored at R3 at location 0x20000020

```

13
14     area    mydata, data, readonly
15 test_data
16     dcb 4, 7
17     dcb 2, 3
18     dcb 9, 1
19

```

Figure 1.2: Test Data that provides different values for different test cases

R4 will store the values for different test cases that will be iterated through for every test case. From Figure 1.2, the first value of the BCD value that is 47, because 4 and 7 are stored in separate bytes, the value of 4 is loaded onto R5. Then R4's location is moved up by 1 bit so the current value of R4 is now holding the value of 7. It is then loaded onto R6. R5 and R6 now holds the tens and ones values respectively for the 2-digit BCD value. The tens, in R5, are then stored onto R1 with the ones, in R6, are stored in R2.

```

55
56 test_case
57     ldrb r7, [r1]
58     ldrb r8, [r2]
59
60     mov r9, #0x0A
61     mul r7, r7, r9
62     add r7, r7, r8
63
64     bx lr
65
66     end
67

```

Figure 1.3: Test Case function which contains the algorithmic logic

The Test Case function from Figure 1.3 is then called to perform the algorithmic logic to convert the BCD number into its equivalent 8-bit binary value. In the Test Case function, the value that is stored in R1 (that is 4 being the tens) is loaded onto R7 and the value in R2 (that is 7 being the ones) is loaded onto R8. A dummy register R9 used to hold the value of 10 (represented by hex 0x0A) for multiplication purposes. From the formula

$$\text{Binary} = (\text{tens} \times 10) + \text{ones}$$

R7 is multiplied with R9 to get

$$\text{tens} \times 10$$

Since R7 holds the value 4

$$R7 = R7 \times R9$$

$$R7 = 4 \times 10$$

$$R7 = 40$$

Then R7 is added with R8 which holds the value 7 so

$$R7 = R7 + R8$$

$$R7 = 40 + 7$$

$$R7 = 47$$

The main function is called back. R1, R2 moves up by one bit to prepare for the next test case. R7 which holds our decimal value in 8-bit binary will be stored into R3. Then R3 moves up by one bit.

```
25 Reset_Handler
26     mov r0, #3
27     bl main
28     stop
29     b stop
```

Figure 1.4: Reset Handler function that initialise the loop counter and calls the main function

R0 holds the value 3 so every during iteration as declared in the `Reset_Handler` function from Figure 1.4, so for every iteration of the `loop_tests` from Figure 1.1, R0 is subtracted with 1. R0 is then compared with the value 0. The zero, Z, flag is not triggered so the 'branch if not equal' is triggered so the loop iterates again until R0 reaches 0. Only then will the function exit the loop and terminate the program.

1.3 Execution Result

From the values stored in Test Data, the expected results are

$$47_{10} = 2F_{16}$$

$$23_{10} = 17_{16}$$

$$91_{10} = 5B_{16}$$

We can observe the memory window of our chip to observe whether the values are stored correctly.

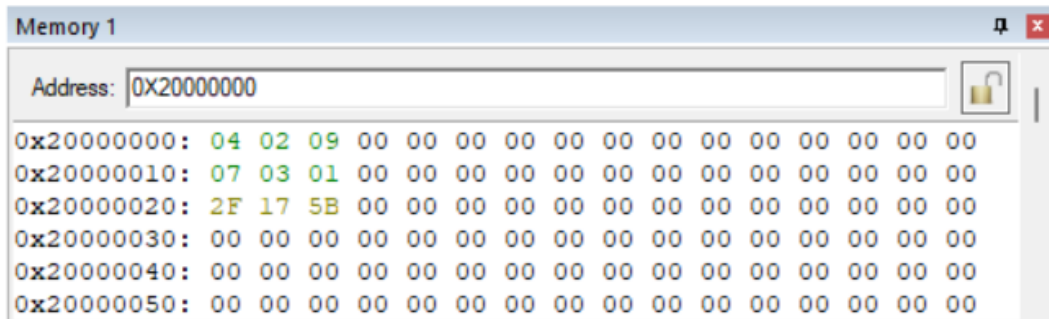


Figure 1.5: Memory 1 viewing memory from address 0x20000000

It is evident with all three cases of 47, 23, and 91, each BCD digits are merged successfully and stored into one register that holds their equivalent 8-bit binary value.

2 ASCII to BCD Conversion

2.1 Description

This assembly program converts a 2-digit ASCII number into a packed BCD format.

2.2 Explanation

```
30
31  main
32      push {lr}
33
34      ldr r1, =0x20000000 ;store tens
35      ldr r2, =0x20000010 ;store ones
36      ldr r3, =0x20000020 ;store data
37      ldr r4, =test_data
38
39  loop_tests
40      ldrb r5, [r4]
41      add r4, #0x01
42      ldrb r6, [r4]
43      add r4, #0x01
44
45      bl test_logic
46
47      strb r5, [r3]
48      add r3, #1
49      add r1, #0x01
50      add r2, #0x01
51      sub r0, #1
52      cmp r0, #0
53      bne loop_tests
54
55      pop {lr}
56      bx lr
57
```

Figure 2.1: Main loop to iterate through different test cases and calls logic function

Below is the memory layout used for this program based on the `main_loop` from Figure 2.1

- ASCII tens digit stored in R1 at location 0x20000000
- ASCII ones digit stored in R2 at location 0x20000010
- Packed BCD output stored in R3 at location 0x20000020

```

13      area    mydata, data, readonly
14
15  test_data
16      dcb 0x35, 0x37
17      dcb 0x39, 0x31
18      dcb 0x37, 0x37
19

```

Figure 2.2: Test Data that provides different values for different test cases

R4 stores the address of `test_data` based on Figure 2.2, which contains three different ASCII values for three different test cases. The first test case contains ASCII digits 0x35 and 0x37, which represent the hexadecimal code for the digits 5 and 7 respectively to represent 57.

During each iteration of the `loop_tests` from lines 39 to 53 based on Figure 2.1, the first ASCII digit, 0x35 is loaded onto R5 and the second ASCII digit. Then R4 moved up it's position by 1-bit so the next ASCII digit loaded onto R6 would be 0x37. R4's address position is moved up by 1-bit again to prepare for the next test case. The values in R5 and R6 are then passed to the `test_logic` function to convert the 2-digit ASCII number into a packed BCD format.

```

57
58  test_logic
59      mov r7, #0x30
60      sub r5, r5, r7 ;5, 0000 0101
61      sub r6, r6, r7 ;7, 0000
62
63      strb r5, [r1]
64      strb r6, [r2]
65
66      lsl r5, #0x04 ;shift 0000 0101 to 0101 0000
67      orr r5, r5, r6 ; logic or 0101 0000 with 0000 0111
68
69      bx lr
70
71  end

```

Figure 2.3: Test Logic function which contains the algorithmic logic

The ASCII values cannot be directly combined into a BCD form because the ASCII digits begin from hexadecimal value of 0x3X, with X being an integer from 0 to 9. Therefore, an algorithmic logic is implemented based on the codes in the `test_logic` function based on Figure 2.3. The first step is to subtract 0x30 from both values in R5 and R6. For this case, R7 will hold the

hexadecimal value of 0x30.

$$\begin{aligned} R5 &= R5 - R7 \\ &= 35_{16} - 30_{16} \\ &= 5_{16} \\ R6 &= R6 - R7 \\ &= 37_{16} - 30_{16} \\ &= 7_{16} \end{aligned}$$

Now R5 contains the tens digit and R6 contains the ones digit. To form a packed BCD number using only 8-bits, the tens digits must only occupy the upper nibble (4-bits) while the ones digit must occupy only the lower nibble. So the tens digit in R5 that is currently 00000101_2 is shifted 4 units to the left so it would be 01010000_2 . So now the value in R5 would hold the hexadecimal value 0x50. The logical OR operation is then performed between R5 and R6.

$$\begin{aligned} R5 &= R5 \text{ ORR } R6 \\ &= 0x50 \text{ ORR } 0x07 \\ &= 0x57 \end{aligned}$$

```
21
22     entry
23     export Reset_Handler
24
25 Reset_Handler
26     mov r0, #3
27     bl main
28     stop
29     b stop
30
```

Figure 2.4: Reset Handler function that initialise the loop counter and calls the main function

The link register then returns the function to the main function in Figure 2.1. The packed BCD result in R5 is then stored into memory through R3. The `test_loop` repeats for the remaining test cases until R0 reaches 0, after which the link register will point back to the `Reset_Handler` function in Figure 2.4

2.3 Execution Result

From the values stored in `test_data`, the expected results are

- Test case 1: Tens = 0x35, Ones = 0x37, Output = 0x57
- Test case 2: Tens = 0x39, Ones = 0x31, Output = 0x91
- Test case 3: Tens = 0x37, Ones = 0x37, Output = 0x77

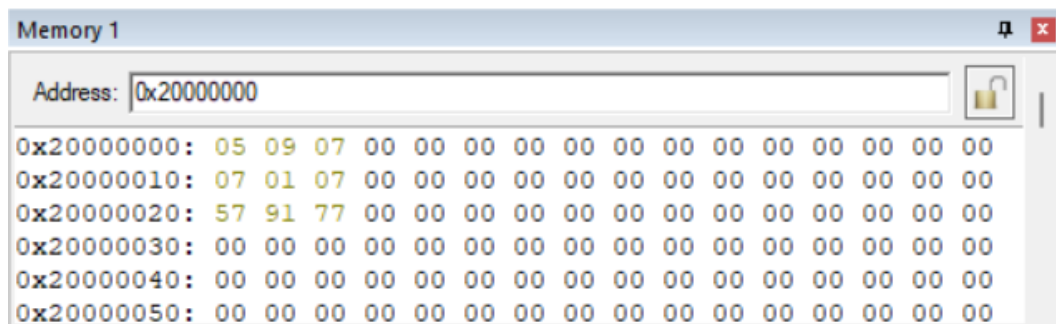


Figure 2.5: Memory 1 viewing memory from address 0x20000000

From the memory window, we get that all our values that are expected are stored correctly in their respective memory locations.

3 Packed BCD to ASCII Conversion

3.1 Description

This assembly program converts a packed BCD byte (8-bits) into it's ASCII representation

3.2 Explanation

```
28
29  main
30      push(lr)
31      ldr r1, =test_data ;loads test data in register
32      ldr r2, =0x20000010 ;stores bcd code for ones
33      ldr r3, =0x20000020 ;stores bcd code for tens
34      ldr r4, =0x20000030 ;stores ascii code for ones
35      ldr r5, =0x20000040 ;stores ascii code for tens
36
37  loop_test
38      ldrb r12, [r1], #1
39
40      bl test_logic
41
42      strb r9, [r2]
43      strb r7, [r3]
44      strb r12, [r4]
45      strb r11, [r5]
46
47      add r2, #0x01
48      add r3, #0x01
49      add r4, #0x01
50      add r5, #0x01
51
52      sub r0, r0, #1
53      cmp r0, #0
54      bne loop_test
55
56      pop(lr)
57      bx lr
58
```

Figure 3.1: Main loop to iterate through different test cases and calls logic function

Below is the memory layout used for this program as declared in the `main` from Figure 3.1 from lines 31 to 35.

- BCD for ones digits stored through R2 at location 0x20000010
- BCD for tens digits stored through R3 at location 0x20000020
- ASCII ones digit stored through R4 at location 0x20000030
- ASCII tens digit stored through R5 at location 0x20000040

```

12
13     area mydata, data, readonly
14 test_data
15     dcb 47
16     dcb 92
17     dcb 57
18

```

Figure 3.2: Test Data that provides different values for different test cases

R1 stores the address of `test_data`, which contains several packed BCD values for testing based on Figure 3.2. The first test case contains 0x47 representing the decimal number 47. The packed BCD byte stores two decimal digits inside one byte so that the upper nibble will store the tens digit and the lower nibble will store the ones digit. Then the first value of R1 being 0x47 is loaded onto R12 to be converted into ASCII.

```

58
59 test_logic
60     mov r6, #10 ;stores decimal 10 into r6
61     udiv r7, r12, r6 ;r7 = 47 / 10 = 4
62     mul r8, r7, r6 ;r8 = 4 * 10 = 40
63     sub r9, r12, r8 ;r9 = 47 - 40 = 7
64
65     mov r10, #0x30
66     add r11, r7, r10 ;r11 = 0x04 + 0x30 = 0x34
67     add r12, r9, r10 ;r12 = 0x07 + 0x30 = 0x37
68
69     bx lr
70
71 end

```

Figure 3.3: Test Logic function which contains the algorithmic logic

The `test_logic` function based on Figure 3.3 involved an algorithm of extracting the tens and ones from 0x47 then to be added with 0x30 to be converted into ASCII code. Let R6 hold the value of 10₁₀. R12 is divided with R6 to extract the tens digit. The extracted tens digit is stored in R7.

$$\begin{aligned}
 R7 &= R12 \div R6 \\
 &= 47 \div 10 \\
 &= 4
 \end{aligned}$$

Then, the tens digit is multiplied with 10 again so the value can be subtracted from the packed BCD to extract the ones only. For this to happen, the multiplication of the tens digit with 10 is stored in R8. Then the subtraction between the packed BCD in R12 and the multiplied tens digit with 10 in R8 is stored in R9. Hence, R9 will store our extracted ones value from the packed

BCD.

$$\begin{aligned}R8 &= R7 \times R6 \\&= 4 \times 10 \\&= 40 \\R9 &= R12 \div R8 \\&= 47 - 40 \\&= 7\end{aligned}$$

The extraction of the tens and ones are stored in R7 and R9 respectfully. Let R10 store the hexadecimal digit 0x30. This value would be added to the values of tens and ones in R7 and R9 to be converted into ASCII code in hexadecimal format.

$$\begin{aligned}R10 &= 30_{16} \\R11 &= R7 + R10 \\&= 4_{16} + 30_{16} \\&= 34_{16} \\R12 &= R9 + R10 \\&= 7_{16} + 30_{16} \\&= 37_{16}\end{aligned}$$

R11 now stores our tens digit while R12 now stores the ones digit for the ASCII code in hexadecimal form. Then the link register points back to the `loop_test` branch for the next test case.

```
19      area mycode, code, readonly
20
21      entry
22      export Reset_Handler
23 Reset_Handler
24      mov r0, #3
25      bl main
26      stop
27      b stop
28
```

Figure 3.4: Reset Handler function that initialise the loop counter and calls the main function

Only when the loop count in R0 reaches 0 the `loop_test` branch will be exited from and the

link register will point back to the `Reset_Handler` function from Figure 3.4 and terminate the program.

3.3 Execution Result

From all the test cases, our expected value for each packed BCD is

- 47, Tens: 0x34, Ones: 0x37
- 92, Tens: 0x39, Ones: 0x32
- 57, Tens: 0x35, Ones: 0x37

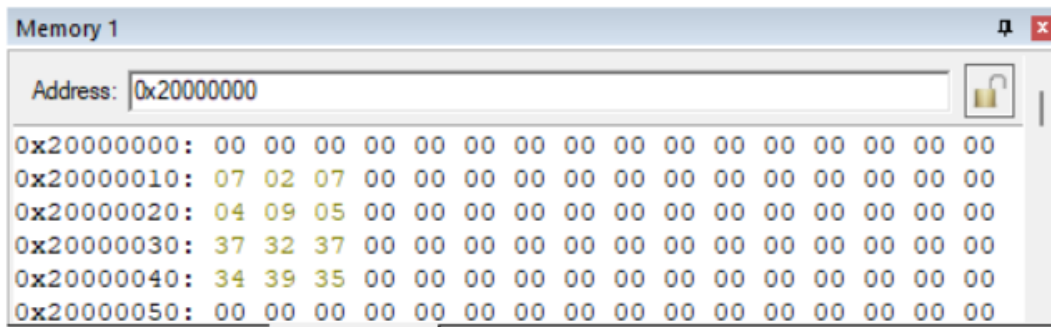


Figure 3.5: Memory 1 viewing memory from address 0x20000000

From the memory window, we observe that all BCD conversion to ASCII representation has been done successfully.

4 Binary to Packed BCD Conversion

4.1 Description

This program converts an 2-digit 8-bit binary number with the range of 0 to 99 into a packed BCD format

4.2 Explanation

```
28
29 main
30     push {lr}
31
32     ldr r1, =test_data      ; input pointer
33     ldr r7, =0x20000010     ;to store (and show) input
34     ldr r2, =0x20000020     ; output pointer
35
36
37 loop_test
38     ldrb r3, [r1]          ; load value
39     add r1, r1, #0x01
40     strb r3, [r7]
41     add r7, r7, #0x01
42
43     bl test_logic          ; process
44
45     strb r5, [r2], #1      ; store BCD result
46
47     sub r0, r0, #1
48     cmp r0, #0
49     bne loop_test
50
51     pop {lr}
52     bx lr
53
```

Figure 4.1: Main loop to iterate through different test cases and calls logic function

Below is the memory layout to show the register's location to store (and show) the input and the output based on codes from line 32 to 34 from Figure 4.1.

- 8-bit binary number is stored through R7 at location 0x20000010
- Packed BCD output is stored through R2 at location 0x20000010

```

12
13     area mydata, data, readonly
14
15 test_data
16     dcb 99, 45, 87    ; 99, 45, 87
17

```

Figure 4.2: Test Data that provides different values for different test cases

R1 will store the test datas that are to be used for different test cases. Based on Figure 4.2, taking the first test case of $R1 = 99_{10}$, the value of R1 is loaded onto R3 to be converted into a packed BCD format. R3 will now hold the $99_{10} = 63_{16}$ and is stored in R7 to be stored as the input value for visualisation.

```

54
55 test_logic
56     mov r4, #10
57
58     udiv r5, r3, r4    ; r5 = tens
59     mul  r6, r5, r4    ; r6 = tens*10
60     sub  r6, r3, r6    ; r6 = ones
61
62     lsl  r5, r5, #4    ; shift tens
63     orr  r5, r5, r6    ; combine ? BCD
64
65     bx  lr
66
67     end

```

Figure 4.3: Test Logic function which contains the algorithmic logic

The `test_logic` function is called for the algorithmic conversion based on Figure 4.3. Let R4 hold the value of 10_{10} . R3 is divided with R4 to extract the tens digit from the decimal value in R3 that is stored in R5 while ignoring the remainder from the division. Then the extracted tens digit is multiplied with R4 to get the decimal value without the ones digit that is stored in R6. This value is then subtracted from the decimal value in R3 to extract the ones digit which is later

then stored in R6.

$$\begin{aligned}R5 &= R3 \div R6 \\&= 99 \div 10 \\&= 9 \text{ remainder } 9 \\R6 &= R5 \times R4 \\&= 9 \times 10 \\&= 90 \\R6 &= R3 - R6 \\&= 99 - 90 \\&= 9\end{aligned}$$

Now R5 and R6 now store the tens and ones digit respectfully. Because in a packed BCD format for 2-digit numbers, the upper nibble of the 8-bit binary number will store the tens while the lower nibble will store the ones digit. Since the tens digit is represented as a single digit as $9_{10} = 00001001_2$, to get 1001 from the lower nibble to the upper nibble, the value in R5 is shifted 4 units to the left to get from 00001001_2 to 10010000_2 . Now the tens are in the upper nibble of the 8-bit binary digit. This value then undergoes a logical OR with R6 that currently holds the ones digit with value 00001001_2 then stored in R5. The resulting value in R5 is now $10011001_2 = 99_{16}$ which now represents a packed BCD format. The link register points back to the main function. The value in R5 is stored in R2 and the loop prepares for the next test.

```
18      area mycode, code, readonly
19
20      entry
21      EXPORT Reset_Handler
22
23 Reset_Handler
24      mov r0, #3                ; number of test cases
25      bl main
26      stop
27      b stop
28
```

Figure 4.4: Reset Handler function that initialise the loop counter and calls the main function

Only when R0 reaches the value 0 will then the `loop_test` be exited and the link register will point back to the `Reset_Handler` function in Figure 4.4.

4.3 Execution Result

For all the test cases, our expected results are:

- Test case 1: 8-bit binary digit = $01100011_2 = 63_{16} = 99_{10}$, Packed BCD = 99
- Test case 2: 8-bit binary digit = $00101001_2 = 2D_{16} = 45_{10}$, Packed BCD = 45
- Test case 3: 8-bit binary digit = $01010111_2 = 57_{16} = 87_{10}$, Packed BCD = 87

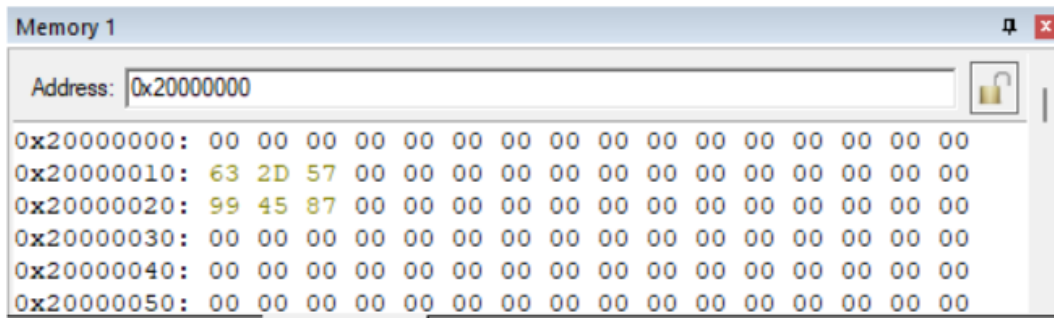


Figure 4.5: Memory 1 viewing memory from address 0x20000000

From the memory window, we observe that all our expected results came out as expected for all test cases. Therefore, the algorithm is correct.

The full source codes that are used in this assignment is uploaded online through GitHub (Khairin, 2025)

References

Khairin, A. (2025). Github - akkzacam/kie2010-assignment-1. *GitHub*. Retrieved April 25, 2026, from <https://github.com/akkzacam/KIE2010-Assignment-1.git>